Advanced Database Systems Portfolio: ValPay

Full Name: Valentine Efagene (MIVA ID: 301921385)

Miva Email Address: v.efagene1385@miva.edu.ng

Course: MIT 8103 – Advanced Database Systems

Assessment: Continuous Assessment (September 2026 cohort) – Portfolios 1–5 and Final Documentation; Case study: ValPay

1. GITHUB REPOSITORY AND SUBMISSION

All code, scripts, evidence and this document are in a public GitHub repository (no access request needed): https://github.com/Valentine-Efagene/advanced-database-portfolio
Running ./scripts/Efagene_Valentine_301921385_InitAndEvidence.sh rebuilds the PostgreSQL database in Docker and regenerates the evidence. Key screenshots are in the appendix.

2. SELECTED ORGANISATION

I chose ValPay, a Nigerian fintech platform offering NGN wallets, peer-to-peer (P2P) transfers, top-ups and auditable transaction history. Retail customers move money, while compliance, finance and support staff need traceability by transfer reference, ledger integrity and KYC (Know Your Customer) status. The same scenario runs through all five portfolios.

3. SUMMARY OF ALL FIVE PORTFOLIOS

Portfolio 1: Database design and modelling

I modelled ValPay relationally with customer, wallet_account, wallet_transaction, ledger_posting, transfer_detail, beneficiary and transaction_audit tables. The ER diagram (Figure 1) shows the one-to-many links between customers, wallets and transactions. The schema enforces non-negative balances, unique transfer references, idempotency keys and no self-transfers. My normalisation notes justify 1NF to 3NF, with the running balance as the one controlled exception.

Portfolio 2: Query processing and optimisation

I wrote six reporting and operational queries: wallet statements, daily volume, reference lookup, low-balance alerts, failed and pending queues, and ledger reconciliation. EXPLAIN showed that Q3, which filters transactions by a reference prefix, ran a sequential scan over wallet_transaction. After I added idx_transaction_reference, the plan changed to a bitmap index scan (Figures 2 and 3), which speeds up support staff tracing VP-TXN references. Q1 already used idx_ledger_account on its join path.

Portfolio 3: Transactions and concurrency

I implemented the P2P transfer as a single PostgreSQL transaction covering the balance check, transaction insert, paired ledger postings and audit entry. An overdraft attempt updated zero rows and rolled back, leaving no trace (Figure 4), while a 2,500 NGN transfer committed with balanced ledger postings. For concurrency control, I ran two payouts against the same wallet at once: session A held a SELECT ... FOR UPDATE row lock, and session B waited on that lock for about nine seconds until A committed. B then read A's committed balance, so both debits applied with no lost update (Figure 5).

Portfolio 4: NoSQL and advanced data models

I stored semi-structured KYC documents, activity events and fraud signals in MongoDB while keeping the ledger in PostgreSQL. KYC payloads vary by document type and events are high-volume and append-only, which suits documents better than wide SQL tables. JSON Schema validation rejects invalid values, compound indexes serve the pending KYC queue, and aggregation pipelines summarise fraud hits (Figure 6). Documents link back to PostgreSQL through customer_id and the transaction reference, without copying any ledger amounts into MongoDB.

Portfolio 5: Distributed and cloud database exercise

I designed a reference production architecture (Figure 7): Multi-AZ Amazon RDS PostgreSQL with synchronous replication for ledger writes, read replicas for reporting, MongoDB Atlas for KYC and events, and Redis for idempotency keys. I based it on vendor documentation and did not provision it; what I ran locally is the Docker Compose stack behind Portfolios 1 to 4. In CAP terms, ledger writes favour consistency, while analytics can accept eventually consistent reads. I also outlined sharding activity_events by customer_id and TTL-based retention for NDPA compliance.

4. INDIVIDUAL REFLECTION

Design choices

I kept wallet balances and ledger postings in PostgreSQL because ValPay needs ACID guarantees and reconcilable double-entry rules. I moved variable KYC payloads, high-volume activity and fraud analytics to MongoDB, where a flexible schema and event-style storage fit better. I used surrogate keys plus unique business references (reference and idempotency_key) to mirror how payment APIs prevent duplicate debits. Storing balance on wallet_account was a deliberate read optimisation, but I only update it inside the Portfolio 3 transfer transaction so it cannot drift from the ledger.

Challenges I faced

- Making Q3 fast: I initially assumed a prefix filter on reference would use a plain B-tree index the way an equality filter does. Repeated EXPLAIN runs confirmed the bitmap index plan.

- Linking two stores without creating two ledgers: I had to resist putting monetary amounts into MongoDB. I kept customer_id and reference as the only join keys and left every debit and credit in PostgreSQL.
- Staging a real lock wait: my first concurrency run raced, because container start-up delayed session A and session B ran first. I fixed it by waiting on database state (A holding the lock, B waiting on it) instead of fixed sleeps.
- Honest scope for cloud high availability: I could not justify the cost of running Multi-AZ RDS and Atlas for coursework, so I separated what I validated locally from what I researched and diagrammed as a production target.

Lessons learned

Polyglot persistence works when there is a hard line: strong consistency wherever money moves, and eventual consistency only where the business can accept lag. ER modelling, index tuning and transaction design reinforce each other, so I stopped treating them as isolated portfolio tasks.

5. REFERENCES

- Amazon Web Services, Inc. (2026). Amazon RDS user guide. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/ (accessed 13 November 2026).
- Federal Republic of Nigeria. (2023). Nigeria Data Protection Act 2023. National Assembly of the Federal Republic of Nigeria.
- Miva Open University. (2026). MIT 8103 Advanced Database Systems: Continuous assessment brief (September cohort). Course materials on the learning management system.
- MongoDB, Inc. (2026). MongoDB documentation. https://www.mongodb.com/docs/ (accessed 13 November 2026).
- PostgreSQL Global Development Group. (2026). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ (accessed 13 November 2026).
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill.

6. AI USE DECLARATION

I used Cursor AI to scaffold the project folders, suggest SQL and MongoDB patterns, render the Portfolio 5 architecture diagram and polish the wording of this document. I made the ValPay design decisions, ran the evidence scripts myself, and verified balances, rollback and locking behaviour, EXPLAIN output and MongoDB query results against my local databases. I corrected wording that implied cloud resources I had not deployed, and I take responsibility for the accuracy of the submission.

APPENDIX: SELECTED EVIDENCE

Screenshots from my local runs. Full logs and further screenshots are in each portfolio's evidence folder on GitHub.

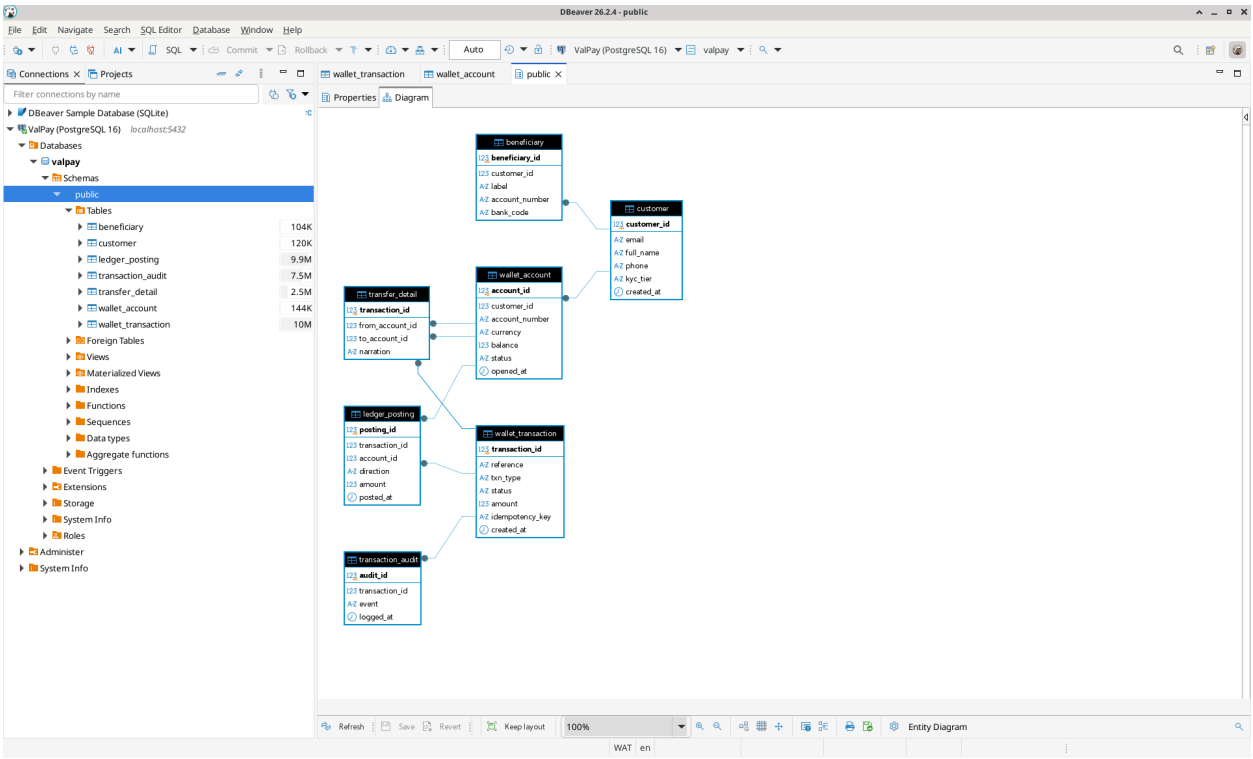


Figure 1: Entity-relationship diagram for the ValPay schema (Portfolio 1)

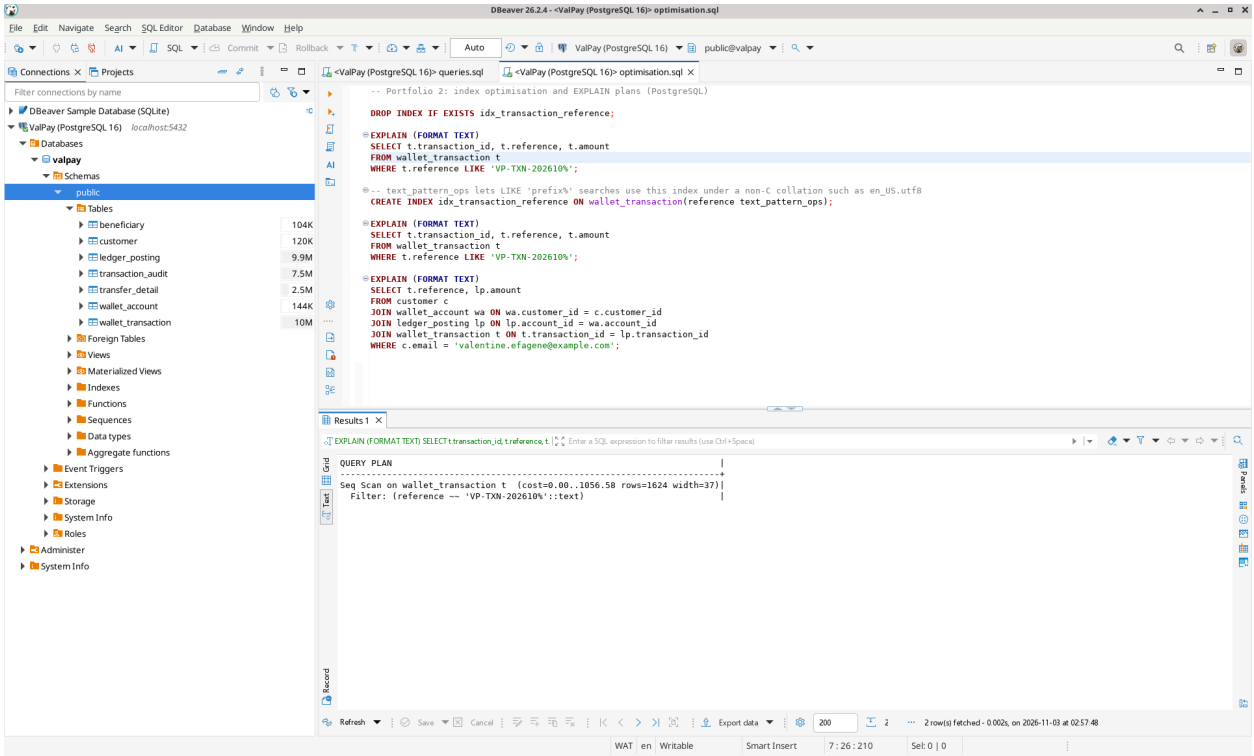


Figure 2: Q3 plan before the index, showing a sequential scan (Portfolio 2)

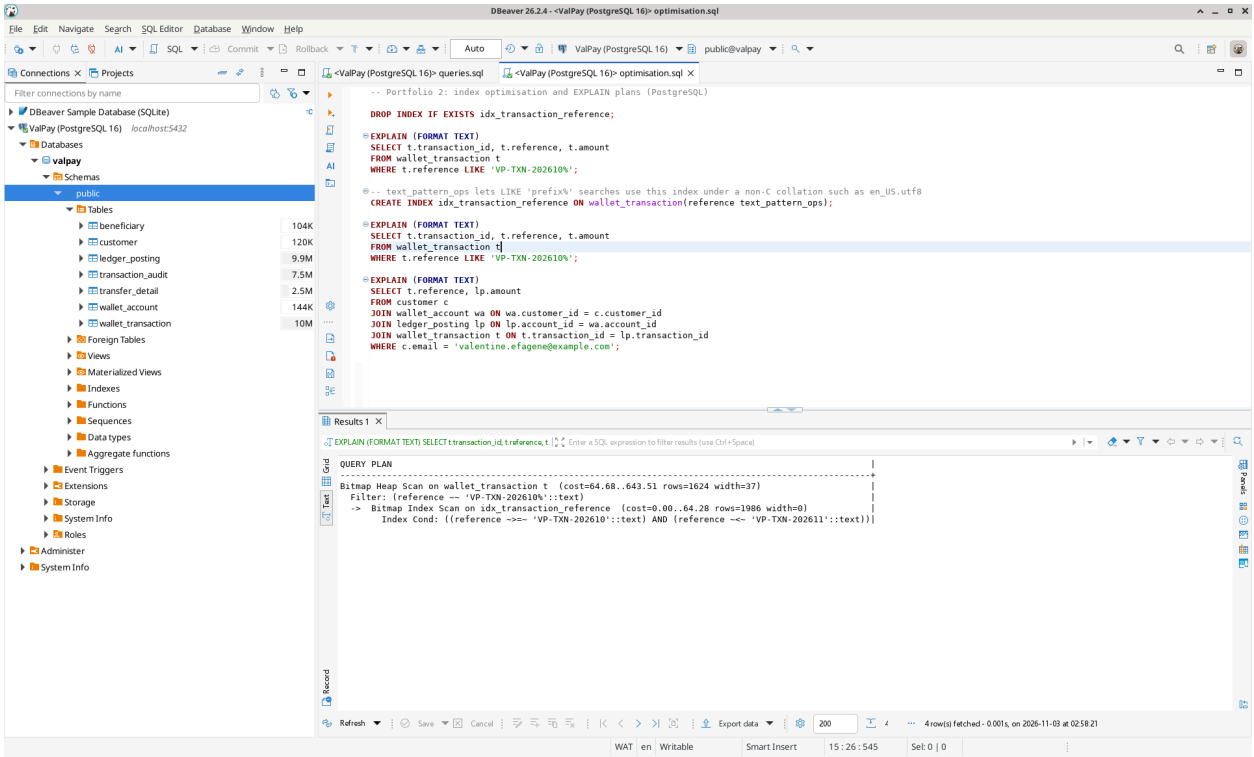


Figure 3: Q3 plan after adding idx_transaction_reference, showing a bitmap index scan (Portfolio 2)

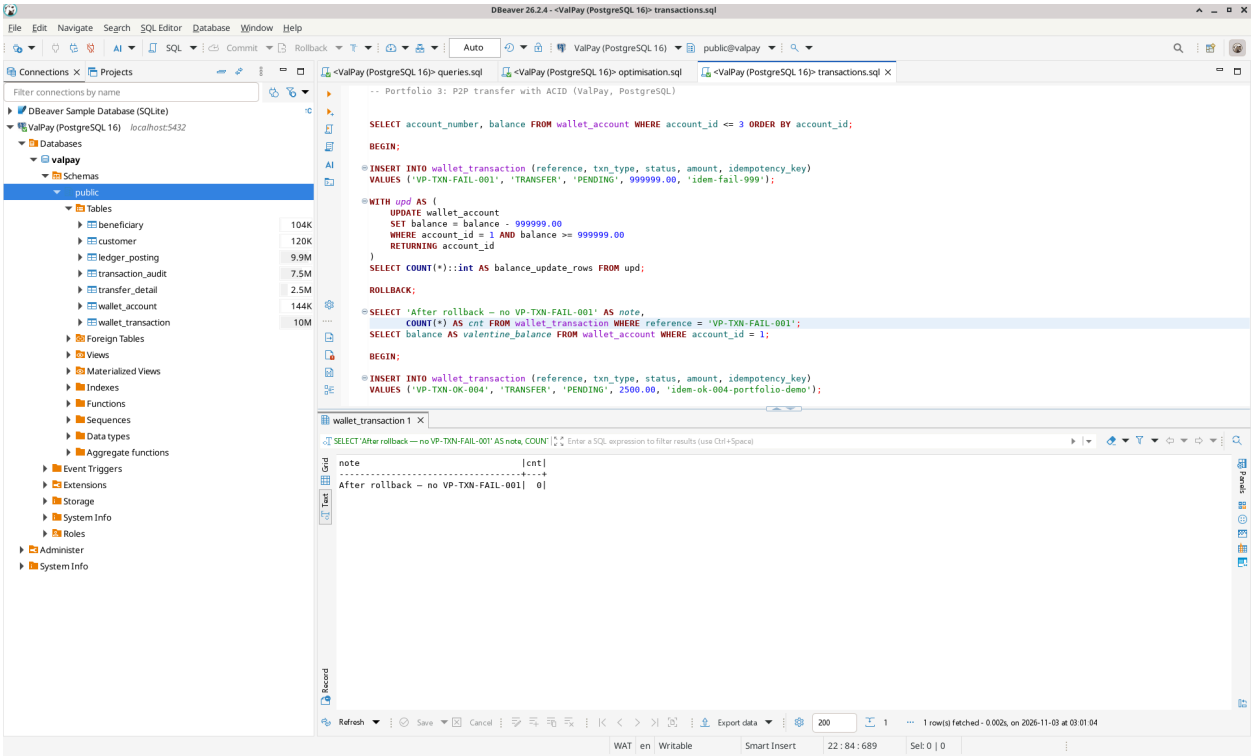


Figure 4: Insufficient-funds transfer rolled back with the sender balance unchanged (Portfolio 3)

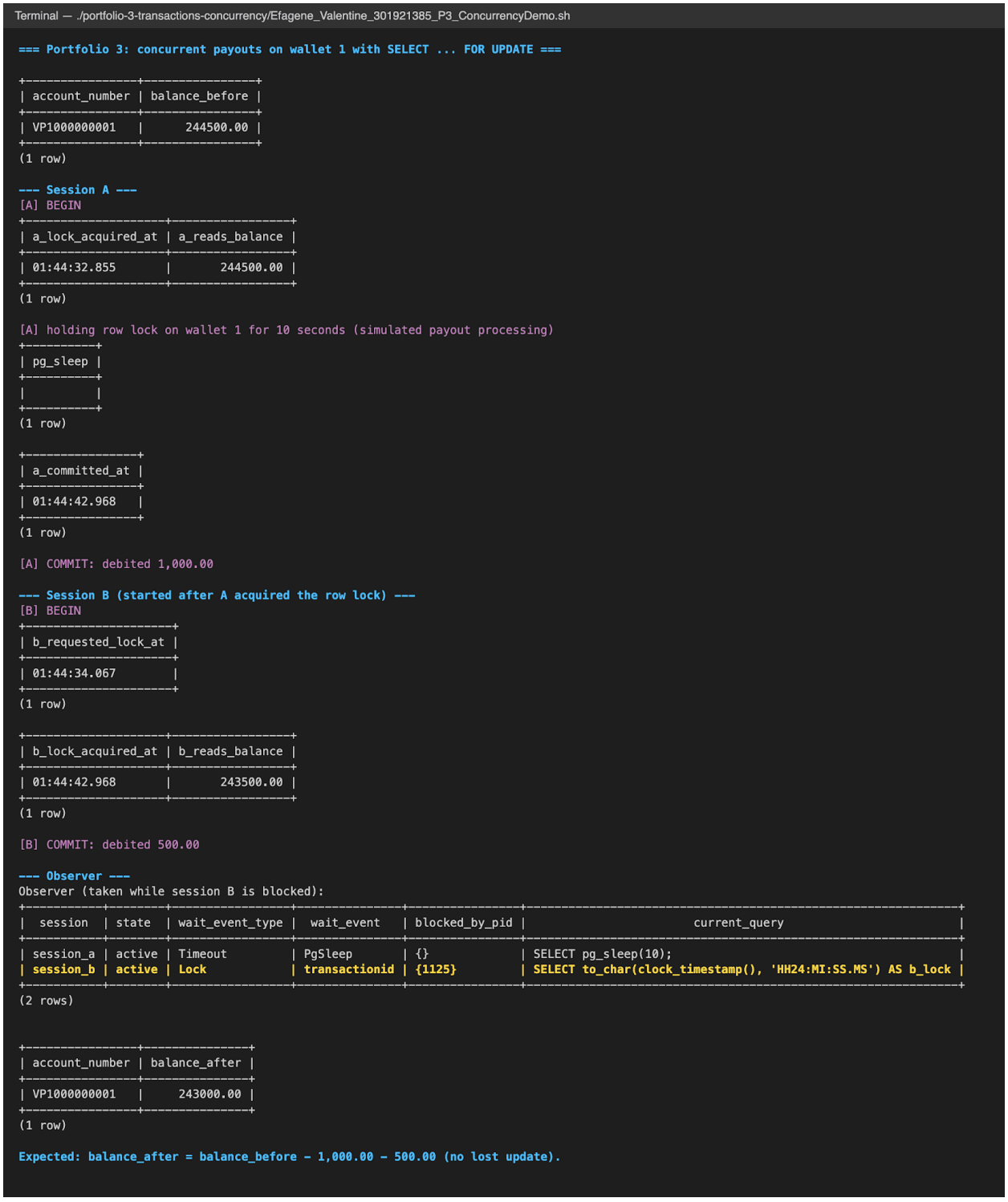


Figure 5: Two-session row-lock demo; session B waits on session A's FOR UPDATE lock, and the final balance reflects both debits (Portfolio 3)

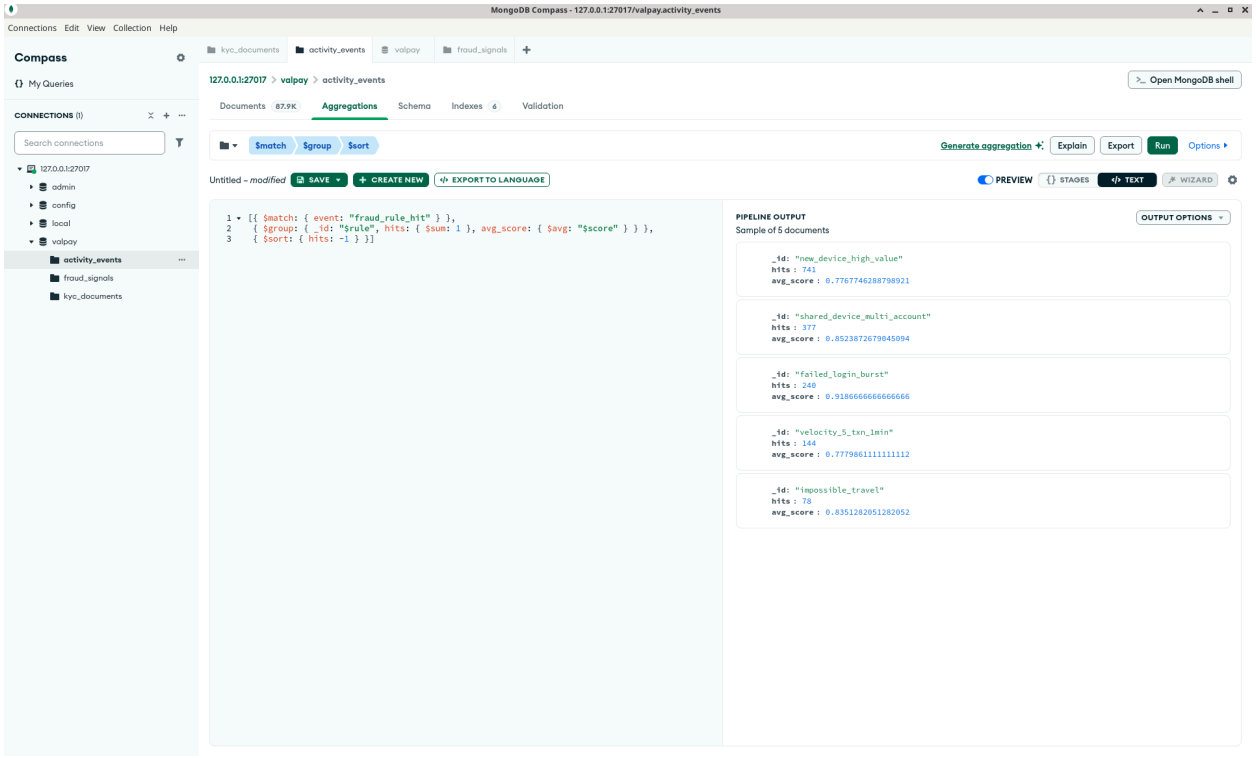


Figure 6: Fraud-signal aggregation in MongoDB Compass (Portfolio 4)

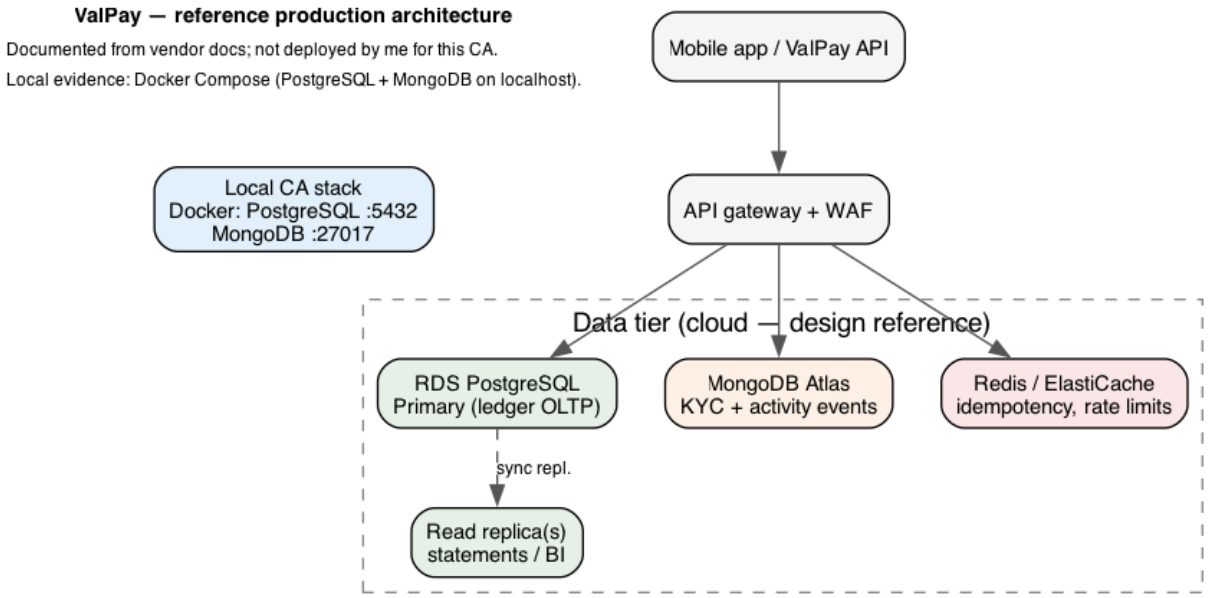


Figure 7: Reference production architecture (design only) and the local Docker stack I ran (Portfolio 5)